How-To Library

Informatica

How to Capture MicroStrategy User Context through Dynamic Data Masking

© Copyright Informatica LLC 2019, 2021. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

MicroStrategy is a business intelligence tool that includes performance management, dashboards, analysis, and reporting capabilities. This article describes how to capture MicroStrategy user context through Dynamic Data Masking.

Supported Versions

- Dynamic Data Masking 9.9 or later

Table of Contents

Overview. 2
Configuring MicroStrategy. 2
Step 1. Enabling User Context for MicroStrategy. 2
Step 2. Clearing the Report Cache for the MicroStrategy Project. 4
Configuring Dynamic Data Masking for MicroStrategy. 7

Overview

To capture MicroStrategy user context through Dynamic Data Masking, you must configure MicroStrategy and Dynamic Data Masking. You configure MicroStrategy so that the name of the user running a report can be included in an SQL query. After you configure MicroStrategy and Dynamic Data Masking, you can take actions on Dynamic Data Masking SQL queries based on user context.

Configuring MicroStrategy

To configure MicroStrategy, perform the following steps:

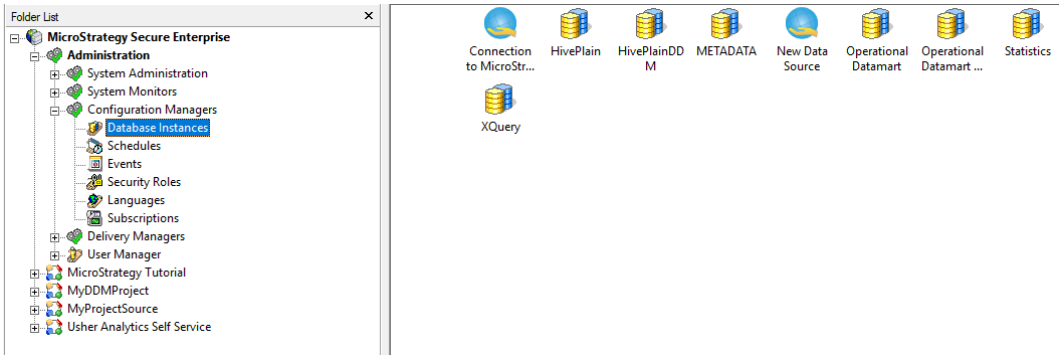
1. Enable User Context for MicroStrategy.
2. Clear the Report Cache for the MicroStrategy Project.

Step 1. Enabling User Context for MicroStrategy

To include the MicroStrategy user name in the SQL query, configure the Database Instance **SQL_HINT** property.

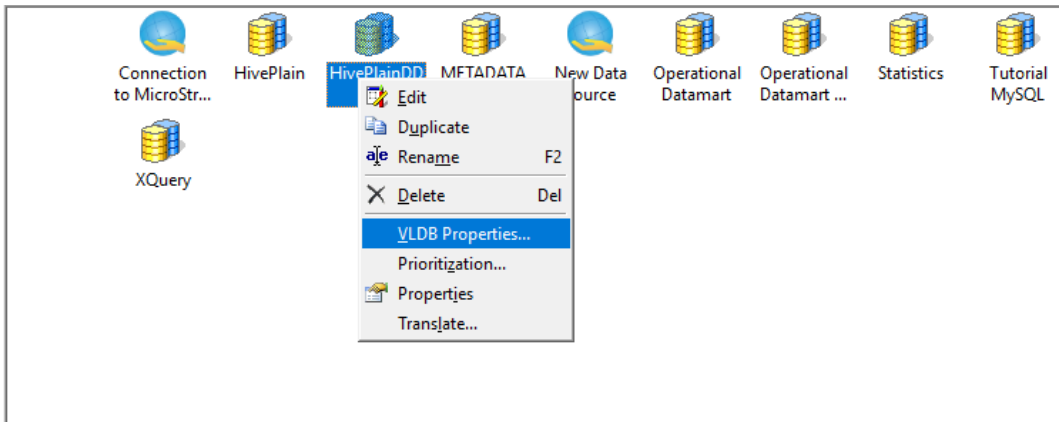
1. Log in to MicroStrategy.
2. Navigate to **MicroStrategy Secure Enterprise** > **Administration** > **Configuration Managers** > **Database Instances**.

The following image shows this view:



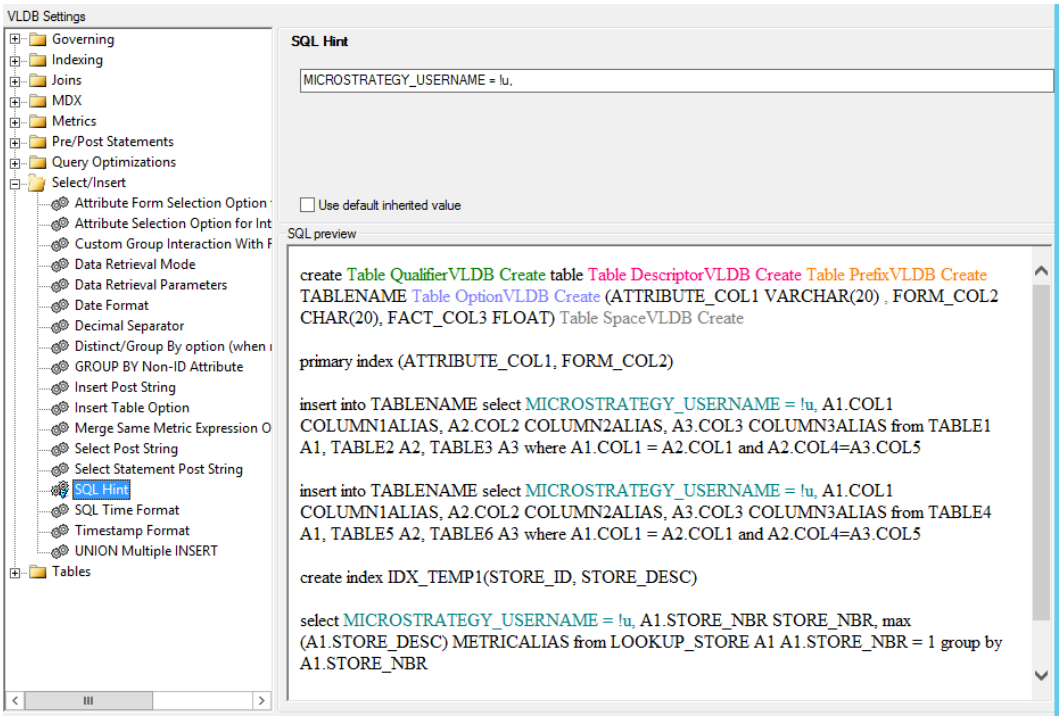
3. Select the Database Instance, and then right-click the instance name.
4. Select **VLDB Properties**.

   The following image shows this view:



   The **VLDB Settings** window appears.

5. Navigate to **Select/Insert** > **SQL Hint**.



6. In the **SQL Hint** field, enter: `MICROSTRATEGY_USERNAME = !u,`.
7. Clear the **Use default inherited value** check box.

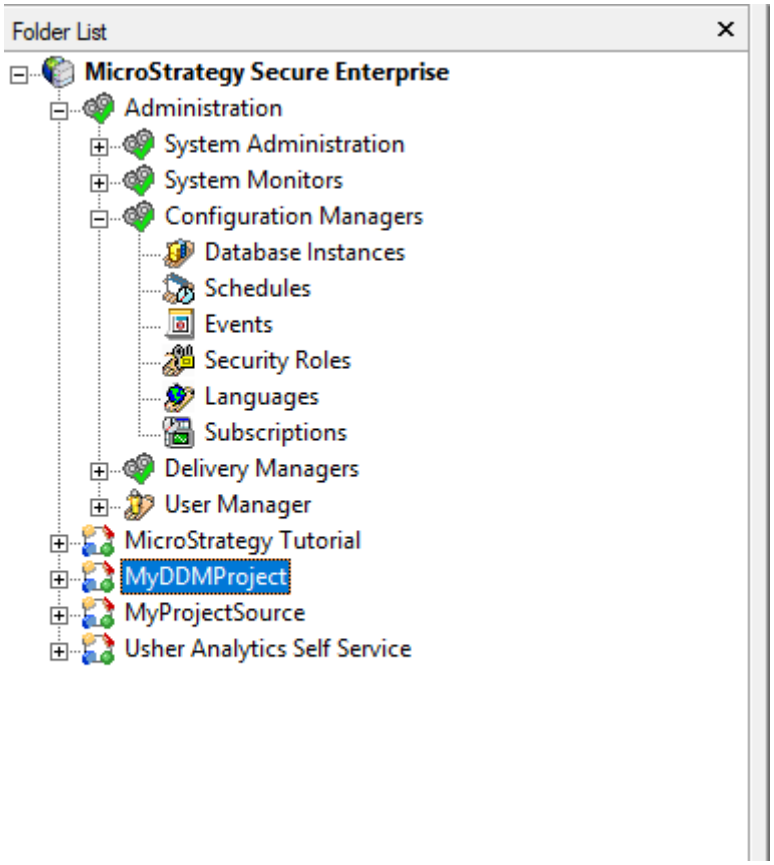
MicroStrategy replaces **!u,** with the MicroStrategy user name.

Step 2. Clearing the Report Cache for the MicroStrategy Project

MicroStrategy caches the results of the report. Clear the cache so that MicroStrategy sends the SQL query to the database each time a report runs.

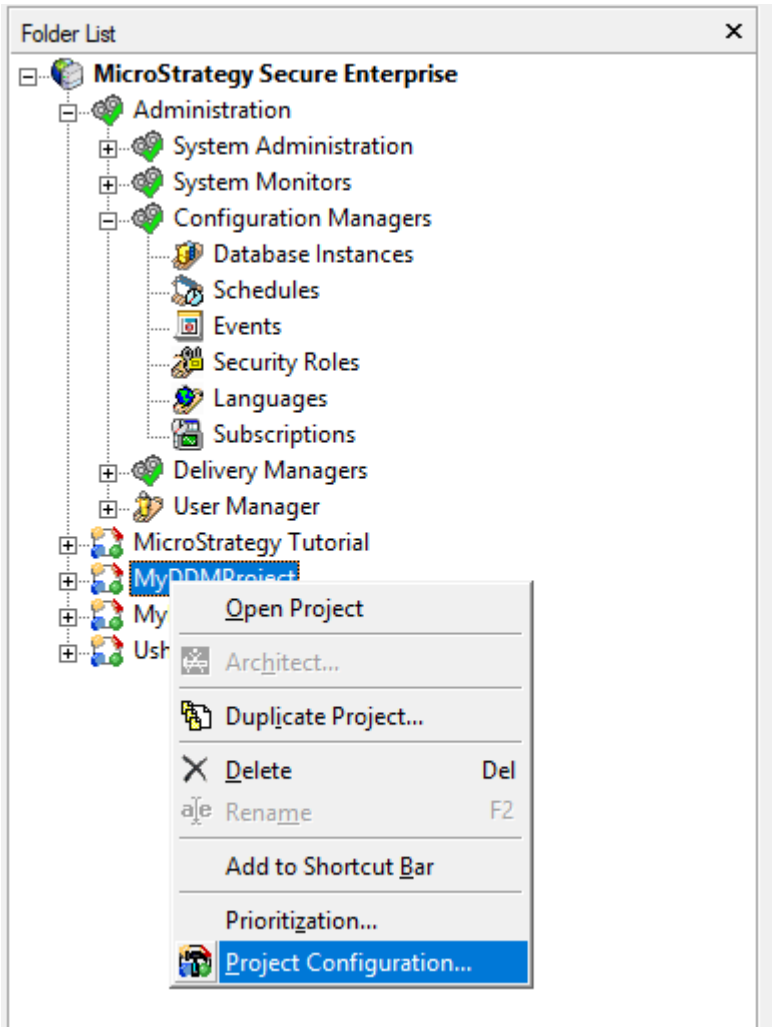
1. Log in to MicroStrategy.
2. Select the project.

The following image shows this view:



3. Right-click the project and select **Project Configuration**.

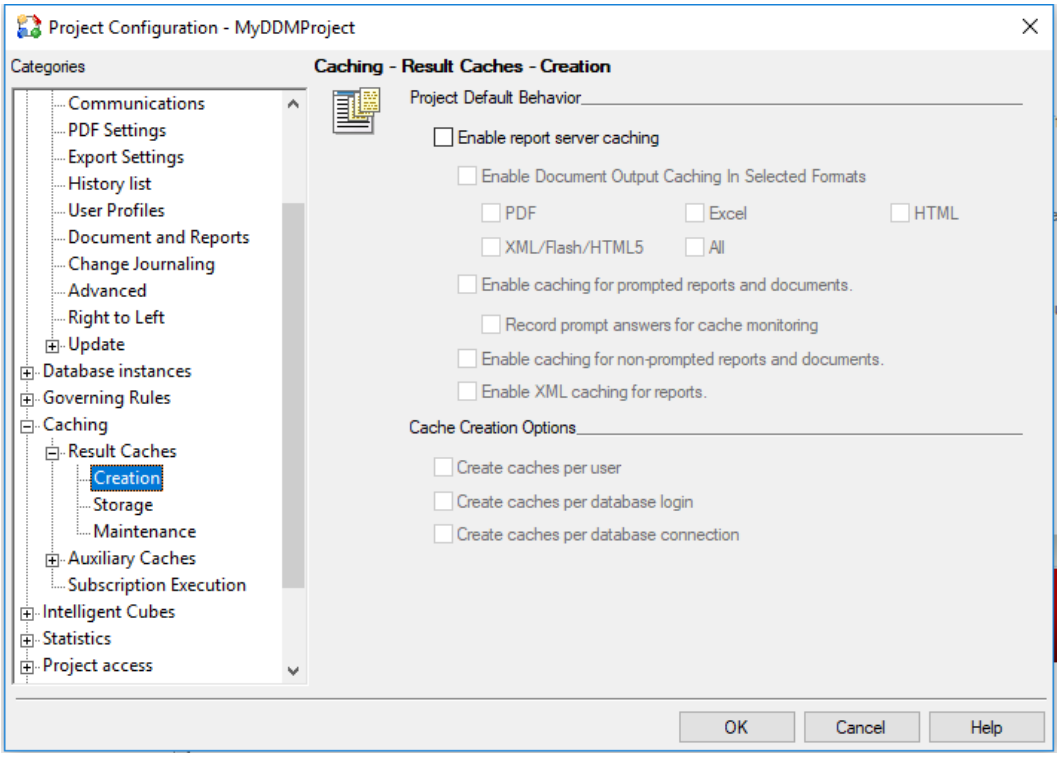
The following image shows this view:



The **Project Configuration** window appears.

4. Navigate to **Caching** > **Caching Results** > **Creation**.

5. Clear the **Enable report server caching** check box.



Configuring Dynamic Data Masking for MicroStrategy

1. Create a Java Action with the following code.

```
/* GetMicroStrategyApplicationUserName.java*/
import com.activebase.content.mask.ContentProcessor;
import com.activebase.contentHandlers.statement.StatementContentHandler;
import com.activebase.logging.TraceFacility;
import com.activebase.rule.RuleContext;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;


public class GetMicroStrategyApplicationUserName {

    private static final String regexforApplicationUserName = "(.*)
(MICROSTRATEGY_USERNAME\\s*=\\s*(.*?),)(.*)";
    private static final String MicroStrategy_USER = "MICROSTRATEGY_USERNAME";
    private static final int flags = Pattern.DOTALL | Pattern.MULTILINE |
Pattern.UNICODE_CASE | Pattern.CASE_INSENSITIVE;
    private static final Pattern patternforApplicationUserName =
Pattern.compile(regexforApplicationUserName, flags);

    public static String execute(RuleContext ctx) {

        StatementContentHandler stmtContenthandler = null;
        if (ctx == null) {
            return null;
        } else {
            ContentProcessor contentProcessor = (ContentProcessor) ctx;
            stmtContenthandler = contentProcessor.getStatementContentHandler();
            if (stmtContenthandler == null) {
```

```
                return null;
            }
        }


        String orginalStatement = stmtContenthandler.getStatement();
        Matcher applicationUserNameMatcher =
patternforApplicationUserName.matcher(orginalStatement);
        if (applicationUserNameMatcher.find()) {
            String microStrategyApplicationUserName =
applicationUserNameMatcher.group(3);
            Map symbolTable = stmtContenthandler.getSymbolTable();
            symbolTable.put(MicroStrategy_USER, microStrategyApplicationUserName.trim());
            String newStatement = applicationUserNameMatcher.group(1) + " " +
applicationUserNameMatcher.group(4);
            return newStatement;
        }

        return null;
    }
}
```

2. Compile the code and create a JAR file. For example, `microStrategyAccelerator.jar`.
3. Configure the JAR file in a Java Action in Dynamic Data Masking.
4. Place the Java Action as the first rule in the rule set.

   After configuring the Java Action, a symbol value for MICROSTRATEGY_USERNAME populates the MicroStrategy user name each time a report runs.
5. Create masking rules based on the symbol MICROSTRATEGY_USERNAME.

Authors

Vamsi Krishna Brahmadevara

Siva Krapa